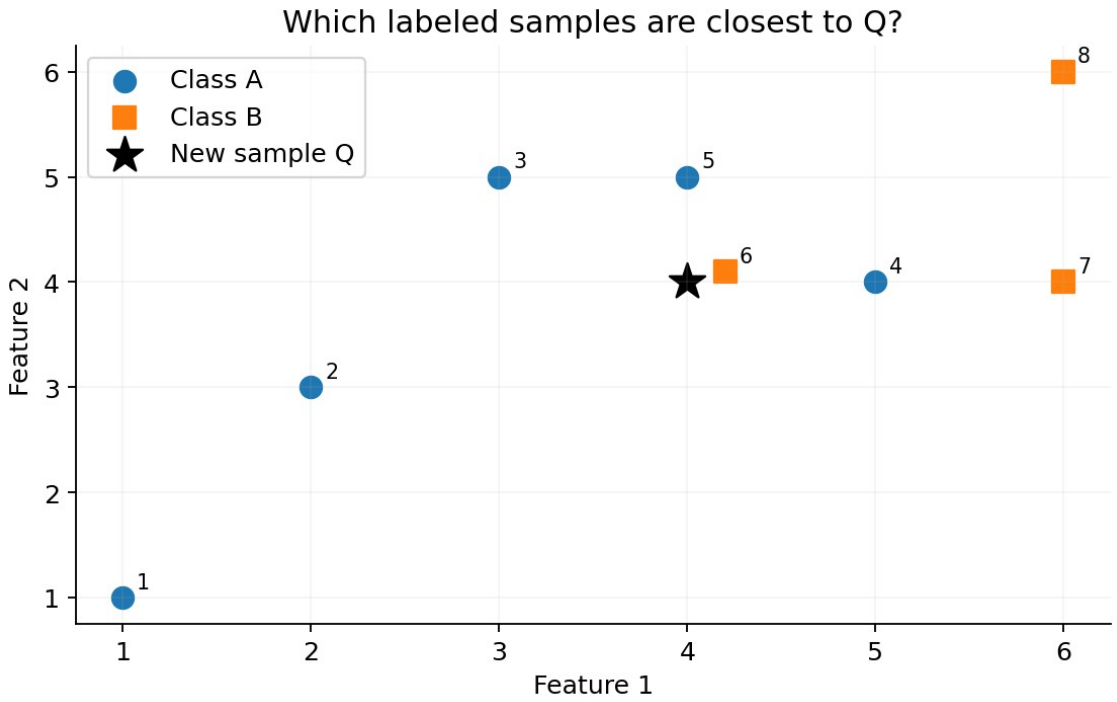
למידה עמוקה | פרק 7 בסילבוס ההתמחות | 30 שעות מסגרת

פרק 7 — סיווג ו־KNN מתקדם

מהפעלת ספרייה למימוש אלגוריתם, חשיבה וקטורית ובדיקת הכללה



איור 1. אותו רעיון מוכר של שכנים קרובים — הפעם לומדים כיצד מממשים אותו בעצמנו.

פרק זה הוא המשך מעמיק לפרקים 4.1–4.4 של נושאי הליבה. במקום לשכפל את ההגדרות, נפרק כאן את השלבים שמבצע המסווג ונבנה אותם בהדרגה בקוד. תחילה נדע להסביר כל שורה, אחר כך נחשב ב־NumPy כמה מרחקים במקביל, ולבסוף נשווה את המימוש שלנו לכלי מקצועי. ההצלחה בפרק אינה נמדדת רק בקבלת תחזית נכונה, אלא ביכולת לזהות שגיאות, לבדוק נכונות ולהצדיק החלטות של מפתח AI.

עוגן: תכנית ההתמחות בלמידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN (30 שעות); חומרי השתלמות 5.1–5.3, כולל מימוש עצמאי ב־NumPy ותרגול Iris. חלוקת התרגול וההעמקה כאן היא הצעה דידקטית מקורית, לא נוסח רשמי של המחוון.

מפת הלמידה — מה נדע לעשות?

- להסביר כיצד אלגוריתם KNN מקבל את מטריצת המאפיינים X, את וקטור התוויות y ואת נקודת השאילתה q.
- לממש חישוב מרחקים, מיון אינדקסים, שמירת תוויות והצבעת רוב בעזרת לולאות, ואז בעזרת NumPy.
- להסביר Broadcasting, את המושג shape ואת משמעות axis בחישוב מרחקים לשאילתה אחת ולמספר שאילתות.
- לקבוע מראש מדיניות שוויון, להבחין בין תווית חזויה לציון הצבעה, ולהשוות הצבעה רגילה להצבעה משוקללת.

- לבדוק התאמת ממדים, תקנון ללא זליגת נתונים, חלוקת Train/Test, בחירת k באמצעות Cross-Validation והשוואה הוגנת.
- לנתח מורכבות חישובית, מגבלות ממדיות ולנסח החלטות מקצועיות בפרויקט גמר.

רצף מוצע ל־30 שעות ההתמחות

תוצר	מיקוד	שעות
טבלת מרחקים + הנמקת סיווג	ריענון סיווג וחישוב ידני	1–5
מימוש שקוף עם לולאות	פיתוח פונקציות Python	6–10
מימוש וקטורי ותיעוד shape	NumPy, Broadcasting וצורות	11–15
בדיקות נכונות והשוואה לספרייה	שוויון, משקולות ובדיקות	16–20
ניסוי Iris מתועד	Pipeline, CV ומדדים	21–25
דוח חקר קצר והגנה בעל־פה	ניתוח עלויות, העברה ותרגול	26–30

1. סיפור פתיחה — מדוע אי אפשר להסתפק ב־fit ו־predict?

דודו בונה מערכת המסייעת לסווג תקלות במעבדת מחשבים. לכל מחשב נשמרים שני מאפיינים מספריים: עומס במעבד ומדד התרעות. לכל מחשב שכבר נבדק קיימת גם תווית: A — תקלה קלה, B — דורש בדיקה נוספת. הדוגמה מומצאת להוראה, ואין להשתמש בה לקבלת החלטות תחזוקה אמיתיות ללא בדיקה מקצועית.

דודו הפעיל KNeighborsClassifier וקיבל תשובה — אבל המורה שאל: כיצד נדע שהמודל בחר את השכנים הנכונים? מה יקרה אם שני מחשבים יהיו באותו מרחק בדיוק? כיצד נבדוק שהנוסחה שהמרנו לקוד מחזירה את אותו המרחק שחישבנו על הנייר? אלה הן שאלות פיתוח תוכנה ומדע נתונים, ולא רק שאלות על פקודות.

$$q = (4,\ 4)$$

לצורך לימוד נשתמש בשמונה דוגמאות אימון ובשאילתה אחת q. כל שורה במטריצה היא מחשב; כל עמודה היא תכונה. התוויות נשמרות בנפרד, משום שאסור למודל להשתמש בתשובה הרצויה בתור מאפיין של הדוגמה החדשה.

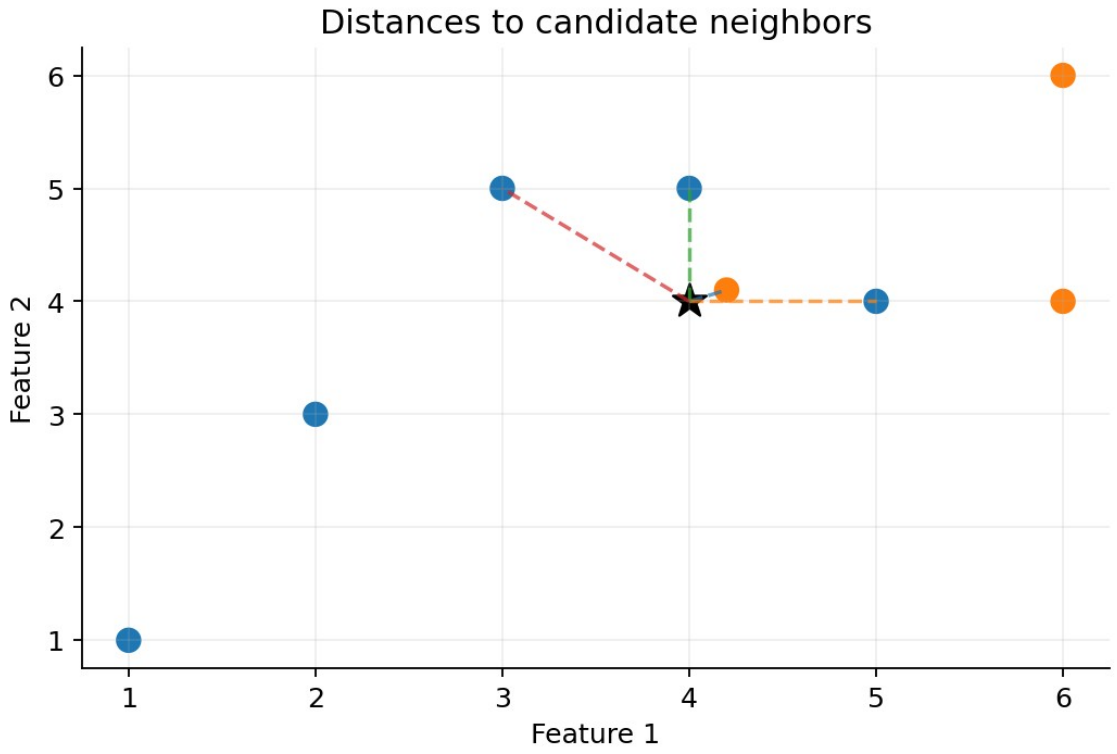
מספר	x_1	x_2	מחלקה
1	1	1	A
2	2	3	A
3	3	5	A
4	5	4	A
5	4	5	A
6	4.2	4.1	B
7	6	4	B
8	6	6	B

2. חזרה קצרה — האלגוריתם בחמש פעולות

תיאור האלגוריתם במילים חשוב לא פחות מקוד. התלמיד צריך להיות מסוגל לספר כיצד הופכת שורה חדשה לתווית חזויה: מודדים את המרחק לכל דוגמת אימון, מסדרים את הדוגמאות לפי מרחק, בוחרים k שכנים, סופרים את התוויות ומחזירים את המחלקה שנבחרה לפי כלל ההכרעה.

$$d(x, q) = \sqrt{\,(x_1 - q_1)^2 + (x_2 - q_2)^2\,}$$

זהו מרחק אוקלידי. כבר למדנו אותו בפרק 1.2; כאן נשתמש בו שוב כדי לתכנן מבנה נתונים נכון. הדבר החשוב הוא שמרחק ותווית שייכים לאותה שורה: כאשר ממיינים מרחקים, אין למיין את התוויות בנפרד.



איור 2. הקווים המקווקווים מדגישים מרחקים של שכנים אפשריים אל השאילתה.

3. חישוב ידני מלא של סיווג אחד

נחשב קודם כמה מרחקים ללא מחשב. לתצפית מספר 6, הווקטור הוא (4.2,4.1). מרחקו מן השאילתה $q=(4,4)$ הוא השורש של סכום ריבועי ההפרשים. לתצפית מספר 4, הווקטור (5,4), ולכן המרחק שווה בדיוק ל-1.

$$d_6 = \sqrt{(4.2 - 4)^2 + (4.1 - 4)^2} = \sqrt{0.05} \approx 0.2236$$

$$d_4 = \sqrt{(5 - 4)^2 + (4 - 4)^2} = 1$$

$$d_5 = \sqrt{(4 - 4)^2 + (5 - 4)^2} = 1$$

לאחר חישוב כל המרחקים נרשום אותם לצד מספר התצפית והתווית. טבלה זו היא כלי בקרה חשוב: היא מאפשרת לאתר טעויות בקוד כגון שימוש בעמודה הלא נכונה או השמטת השורש.

סדר קרבה	תצפית	מרחק משוער	תווית
1	6	0.2236	B
2	4	1.0000	A

סדר קרבה	תצפית	מרחק משוער	תווית
3	5	1.0000	A
4	3	1.4142	A
5	7	2.0000	B
6	2	2.2361	A
7	8	2.8284	B
8	1	4.2426	A

התחזית כאשר k=1 היא B, מפני שהשכן הקרוב ביותר הוא תצפית 6. כאשר k=3, שלוש התוויות הראשונות הן B, A, A ולכן תחזית הרוב היא A. זוהי דוגמה לכך ששינוי k יכול להפוך החלטה גם כאשר כל הנתונים נשארו קבועים.



איור 3. סדר המרחקים לאחר המיון. יש לשמור את אינדקס השורה יחד עם המרחק שלה.

4. ממימוש רעיוני למימוש בפייתון עם לולאות

כדי להבין כיצד המחשב עובד, נתחיל בקוד שקוף ולא בפקודה אחת מוכנה. נשמור מערך של זוגות: המרחק ולידו אינדקס השורה. לאחר המיון נוכל למצוא את התווית מתוך y באותו אינדקס. זהו עיקרון תכנותי חשוב: אינדקס מחבר בין שתי טבלאות שמתארות את אותה תצפית.

```
import numpy as np
X = np.array([[1,1], [2,3], [3,5], [5,4],
              [4,5], [4.2,4.1], [6,4], [6,6]], dtype=float)
y = np.array(["A", "A", "A", "A", "A", "B", "B", "B"])
q = np.array([4.0, 4.0])
rows = []
for i in range(len(X)):
    total = 0.0
    for j in range(X.shape[1]):
        total += (X[i, j] - q[j]) ** 2
    rows.append((np.sqrt(total), i))
rows.sort(key=lambda pair: (pair[0], pair[1]))
print(rows[:3])
```

שאלו את עצמכם: מדוע הלולאה החיצונית עוברת על שורות והפנימית על מאפיינים? מדוע שומרים את i ולא רק את המרחק? מדוע במיון בחרנו צמד (מרחק, אינדקס)? הבחירה באינדקס כקריטריון משני הופכת את סדר השורות לשחזורי במקרה של מרחקים שווים, אבל היא עדיין אינה פתרון מלא לשוויון בהצבעה בין מחלקות.

5. פיתוח פונקציה עם חוזה ברור

לפני מימוש פונקציה נגדיר מה היא מקבלת ומה היא מחזירה. פונקציה לחיזוי אחת מקבלת X בגודל $n \times d$, תוויות y באורך n, נקודת שאילתה באורך d ו־k שלם בין 1 ל־n. תוצאתה היא תווית חזויה. פונקציה מקצועית גם צריכה לזהות קלט לא תקין, אבל לצורך ההיכרות נתחיל במקרה התקין וננסח בדיקות מתאימות בהמשך.

`X.shape = (n, d)` `y.shape = (n,)` `q.shape = (d,)`

```
def distances_one_query(X, q):
    result = np.zeros(X.shape[0], dtype=float)
    for i in range(X.shape[0]):
        total = 0.0
        for j in range(X.shape[1]):
            total += (X[i, j] - q[j]) ** 2
        result[i] = np.sqrt(total)
    return result

print(np.round(distances_one_query(X, q), 4))
```

הפרדת חישוב המרחקים מן ההצבעה מאפשרת לבדוק כל רכיב בנפרד. אם התקבלה תחזית שגויה, אפשר לשאול האם המרחקים שגויים, האם המיון שגוי, או האם ההצבעה שגויה. כך עובדים גם בפרויקט תוכנה גדול: מפרקים בעיה לבדיקות קטנות.

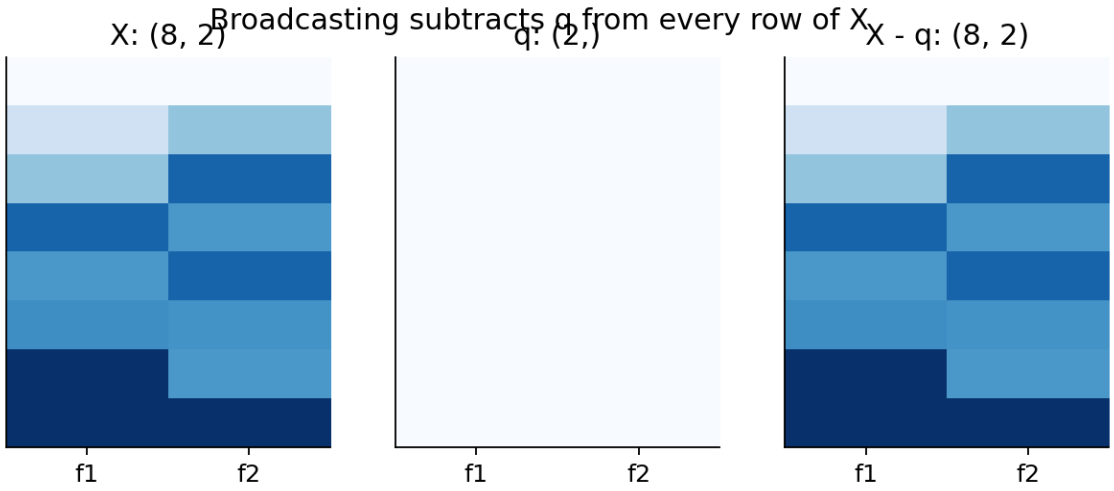
6. NumPy — מדוע לפעמים אפשר להחליף לולאות בפעולת מערכים?

למערך NumPy יש צורה (shape) שמתארת כמה שורות ועמודות יש בו. כאשר X היא מטריצה בעלת שתי עמודות ו־q הוא וקטור בן שני רכיבים, ניתן להחסיר את q מכל שורה ב־X. פעולה זו נקראת Broadcasting: NumPy מתאים מערכים בעלי צורות תואמות ומפעיל את אותה פעולה על קבוצת רכיבים. אין צורך לכתוב בעצמנו לולאה לכל שורה.

```
(8, 2) − (2,)  →  (8, 2)
```

```
differences = X - q
squared = differences ** 2
sums = np.sum(squared, axis=1)
distances = np.sqrt(sums)
print(differences.shape)
print(np.round(distances, 4))
```

אנחנו מסכמים כל שורה בנפרד, משום שכל שורה מתארת תצפית. אם בטעות נשתמש ב־axis=0 ב־axis=1, נקבל שני סכומים — אחד לכל מאפיין — במקום שמונה מרחקים. זו שגיאת תוכן ולא שגיאת תחביר, ולכן חשוב לנבא מראש מה אמורה להיות צורת התוצאה.



איור 4. הפחתת q מכל שורה ב־X ויצירת מטריצת הפרשים. שמירת shape נכונה קריטית למימוש.

7. מיון אינדקסים — מדוע argsort עדיף על sort?

הפקודה np.sort מחזירה את המרחקים כשהם מסודרים, אך בדרך זו נאבד את הקשר אל התווית של התצפית. לעומת זאת, np.argsort מחזירה את האינדקסים שמסדרים את המרחקים. לכן נוכל להשתמש באותם אינדקסים כדי לקרוא מתוך y את המחלקות של השכנים שנבחרו.

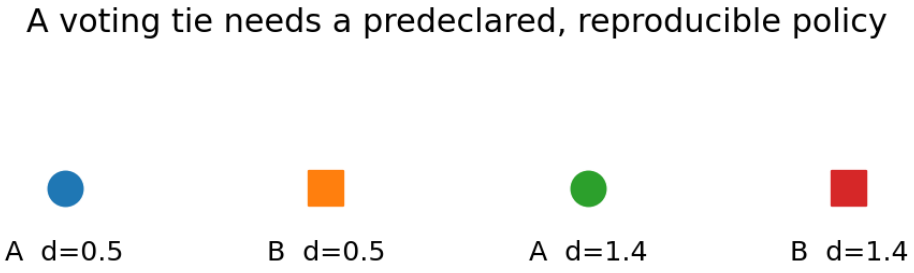
```
order = np.argsort(distances, kind="stable")
k = 3
neighbor_indices = order[:k]
neighbor_labels = y[neighbor_indices]
print(neighbor_indices)
print(neighbor_labels)
```

np.argsort היא אחת הפקודות החשובות בפרק: היא מחזירה מידע על סדר, לא את התוויות ולא את הערכים עצמם. בחרנו stable כדי לשמור על סדרם המקורי של פריטים בעלי מרחק שווה; עדיין צריך להגדיר כיצד מוכרעת הצבעה בין שתי מחלקות עם אותו מספר קולות.

8. הכרעת רוב ומדיניות שוויון — חלק מחוזה האלגוריתם

ב־k אי־זוגי ובבעיה שיש בה שתי מחלקות בלבד, הצבעה ברוב אינה יכולה להסתיים בשוויון במספר הקולות — בתנאי שנבחרו בדיוק k שכנים. אבל עם שלוש מחלקות, או כאשר k זוגי, ייתכן שוויון. נוסף לכך, ייתכן שוויון מרחק בגבול שבין השכן ה־k לבין השכן ה־k+1. אם לא נכתוב כלל מראש, שתי הרצות או שני מימושים עשויים לתת תשובות שונות.

לצורך המימוש הלימודי שלנו נקבע מדיניות מפורשת: במרחקים שווים נשמור תחילה על סדר האינדקסים המקורי; בהצבעת רוב שווה נבחר את המחלקה של השכן הקרוב ביותר מבין המחלקות הקשורות; ואם גם המרחק המינימלי שווה בדיוק, נבחר את שם המחלקה לפי סדר אלפביתי. זוהי מדיניות לימודית שבחרנו לצורך שחזוריות, ולא טענה שהמימוש של scikit-learn מפעיל בהכרח אותו כלל.



איור 5. שתי מחלקות עשויות לקבל מספר קולות זהה. יש לתעד כלל הכרעה לפני הפעלת המודל.

```
  def vote_with_tie_policy(labels, distances):
      classes, counts = np.unique(labels, return_counts=True)
      max_count = np.max(counts)
      tied = classes[counts == max_count]
      if len(tied) == 1:
          return tied[0]
      closest = []
      for label in tied:
          closest.append(np.min(distances[labels == label]))
      min_distance = np.min(closest)
      remaining = tied[np.isclose(closest, min_distance)]
      return sorted(remaining)[0]
```

דוגמה לבדיקה: עבור שכנים עם תוויות A,B,A,B ועם מרחקים 0.2,0.5,0.7,1.1, לכל מחלקה שני קולות. המדיניות שלנו בוחרת A, משום שהשכן הקרוב ביותר מבין המחלקות הקשורות שייך ל־A. אם השכן הקרוב ביותר היה B, הבחירה הייתה משתנה. תלמיד צריך להיות מסוגל להסביר את ההחלטה ללא הרצת הקוד.

9. בניית מסווג אחד קריא ומפורש

נשלב את רכיבי התהליך בפונקציה אחת לצורך הדגמה. נשתמש בפעולת NumPy לחישוב המרחקים, ב־argsort לשמירת אינדקסים ובפונקציית ההכרעה שכתבנו. השמות שנבחרו בקוד מבטאים תפקיד; הם אינם קיצור מסתורי שמחייב לזכור את גוף הפונקציה.

```
def predict_one(X, y, q, k=3):
    if X.ndim != 2 or len(y) != len(X):
        raise ValueError("Expected X: 2D and y: matching rows")
    if q.shape != (X.shape[1],):
        raise ValueError("Query feature count does not match")
    if not 1 <= k <= len(X):
        raise ValueError("k must be between 1 and n")
    distances = np.sqrt(np.sum((X - q) ** 2, axis=1))
    order = np.argsort(distances, kind="stable")[:k]
    return vote_with_tie_policy(y[order], distances[order])

print(predict_one(X, y, q, k=1))
print(predict_one(X, y, q, k=3))
```

שימו לב: עבור ערכי k הנבדקים בדוגמה זו אין שוויון בין מחלקות לאחר ההצבעה. כאשר משווים מימושים שונים על נתונים שיש בהם שוויון גבולי, אין לצפות בהכרח לאותה תוצאה ללא יישור של מדיניות השוויון.

10. מעבר לשאילתות רבות — שלושה ממדים במערך

עד כה חישבנו מרחקים בין שאילתה אחת לבין כל שורות האימון. אבל באפליקציה נחזה בדרך כלל כמה דוגמאות חדשות. נרצה מטריצת מרחקים שבה שורה אחת לכל שאילתה ועמודה לכל דוגמת אימון. כדי לאפשר זאת, נוסיף ממד בגודל 1 לכל אחד מהמערכים וניתן ל־Broadcasting ליצור את כל זוגות ההפרשים.

$$\texttt{Q.shape = (m, d) ; X.shape = (n, d)}$$

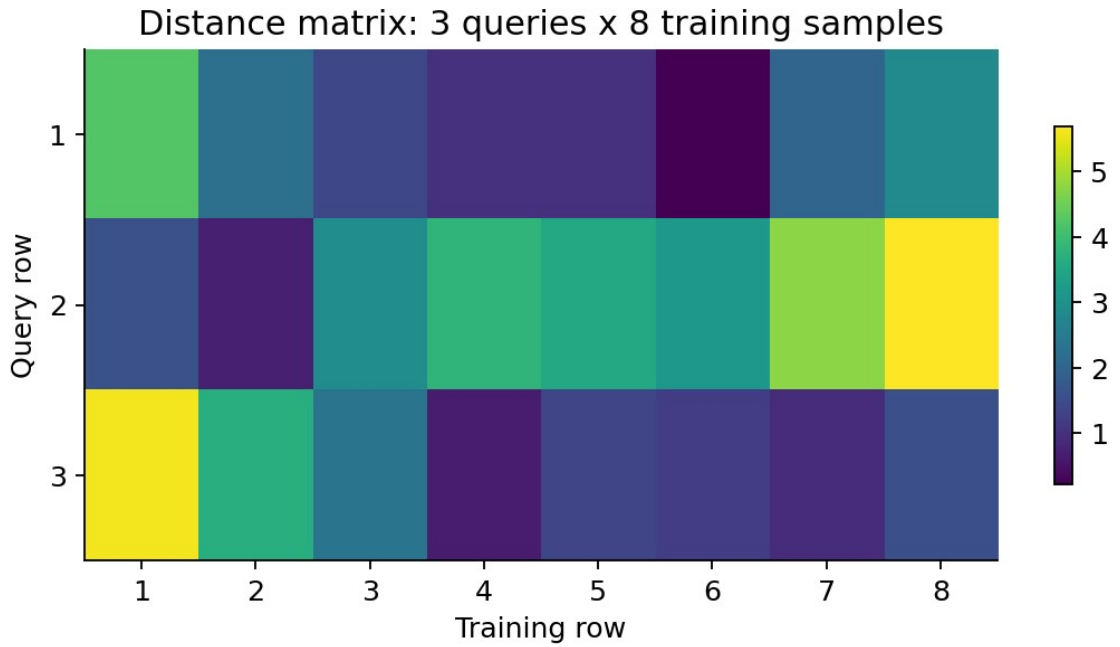
$$\texttt{Q[:, None, :].shape = (m, 1, d)}$$

$$\texttt{X[None, :, :].shape = (1, n, d)}$$

$$\texttt{differences.shape = (m, n, d)} \;\rightarrow\; \texttt{distances.shape = (m, n)}$$

```
Q = np.array([[4., 4.], [1.5, 2.5], [5.3, 4.6]])
all_diffs = Q[:, None, :] - X[None, :, :]
all_distances = np.linalg.norm(all_diffs, axis=2)
print(all_diffs.shape)
print(all_distances.shape)
```

המשמעות של axis=2 כאן שונה מ־axis=1 קודם: במערך החדש ציר 0 מייצג שאילתות, ציר 1 מייצג דוגמאות אימון, וציר 2 מייצג מאפיינים. סכימה על הציר האחרון מחשבת מרחק לכל זוג. אין לשנן axis ללא בדיקת shape.

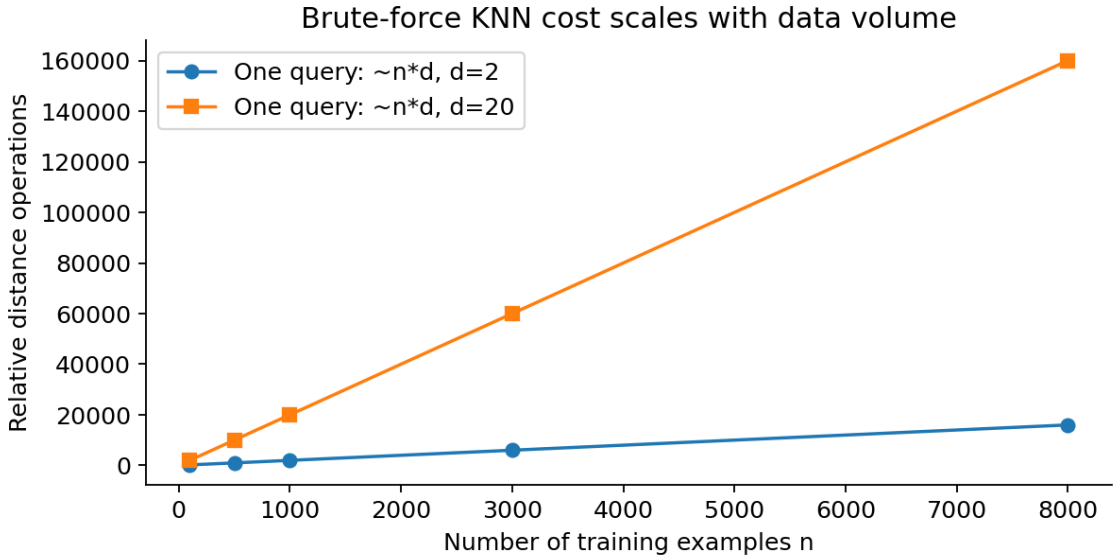


איור 6. מטריצת המרחקים מאפשרת להשוות כמה שאילתות מול כל דוגמאות האימון.

11. האם קוד קצר יותר בהכרח יעיל יותר?

שימוש ב־Broadcasting מקטין את הצורך בלולאות פייתון ויכול להיות מהיר מאוד. אולם עבור m שאילתות, n דוגמאות אימון ו־d מאפיינים, מערך הביניים של ההפרשים עשוי להכיל m×n×d מספרים. כאשר שלושת הגדלים גדולים, צריכת הזיכרון עלולה להפוך לבעיה ממשית. לכן לא נוכל להסיק ש״וקטוריזציה תמיד עדיפה״; נצטרך לשאול גם כמה זיכרון נדרש.

פתרונות אפשריים לפרויקט גדול הם חישוב בקבוצות קטנות (Batches), עבודה עם ספריות שמבצעות חישוב מותאם, או שימוש בשיטות חיפוש שכנים כגון KD-Tree ו־Ball-Tree כאשר תנאי הנתונים מתאימים. אין חובה לממש עצים בפרק זה; חשוב להבין שהמימוש הפשוט אינו הפתרון היחיד.



איור 7. במימוש brute-force, מספר חישובי המרחק גדל עם מספר דוגמאות האימון ומספר המאפיינים.

12. ניתוח מורכבות חישובית ברמת תיכון

נניח שיש n דוגמאות אימון ו־d מאפיינים. כדי לחשב את מרחקה של שאילתה אחת לכל הדוגמאות נבצע בסדר גודל של $n \times d$ פעולות חשבון. כאשר ממיינים את כל n המרחקים, המיון מוסיף עלות בקירוב $n \log n$. לכן עבור המימוש הלימודי עם argsort עלות החיזוי היא בקירוב $O(nd + n \log n)$ לשאילתה אחת, מעבר לעלות האחסון של הנתונים. קיימות שיטות לבחירת k איברים קטנים בלי למיין הכול, אך לא נכניס אותן לליבת המימוש הראשוני.

```
Training storage ≈ O(n d)    |    Distance calculation ≈ O(n d)
Full sort ≈ O(n log n)       |    m-query broadcast memory ≈ O(m n d)
```

יש להבדיל בין זמן חישוב לבין זיכרון. גם אם המחשב מצליח ליצור מערך עצום, אין פירוש הדבר שהפעולה יעילה או מתאימה להפעלה באפליקציה בזמן אמת. שאלה כזו נכנסת לפרק הגנת פרויקט: אילו מגבלות צפיתם, ואיזה שינוי עשיתם כשנפח הנתונים גדל?

13. הצבעה משוקללת לפי מרחק

במנגנון הצבעת רוב כל אחד מן השכנים הנבחרים מקבל קול אחד, גם אם אחד קרוב מאוד ואחר רחוק יותר. בשיטה משוקללת ניתן לתת לשכן קרוב משקל גבוה יותר, למשל באמצעות משקל הפוך למרחק. כך מקרה שבו יש שני קולות למרחקים גדולים וקול אחד קרוב מאוד עשוי להתהפך.

$$w_i = 1 / d_i \qquad (d_i > 0)$$

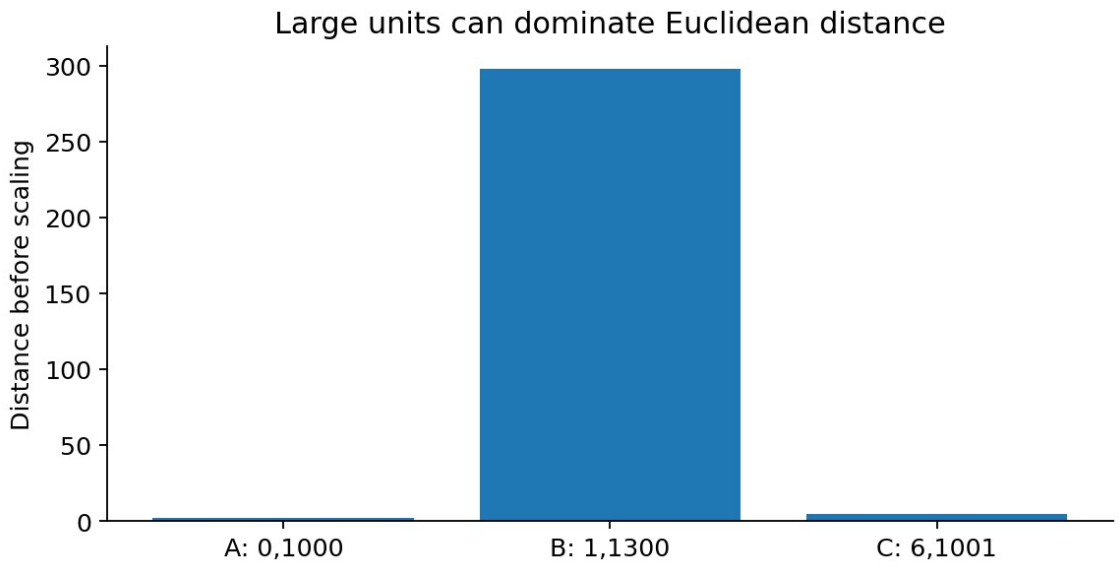
כאשר מרחק שווה לאפס, הביטוי $1/d$ אינו מוגדר. לכן יש צורך במדיניות נפרדת: אם יש שכנים זהים לשאילתה, ניתן להכריע רק ביניהם על פי התוויות שלהם, במקום לחלק באפס. בספריית scikit-learn קיימת אפשרות weights="distance", אך היא מטפלת במקרים מיוחדים לפי המימוש שלה; אין לייחס לה באופן אוטומטי את כללי המימוש הלימודי שלנו.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3, weights="distance")
model.fit(X, y)
print(model.predict(q.reshape(1, -1)))
```

חשוב לא לבלבל משקל בהצבעת KNN עם משקל שנלמד ברגרסיה או ברשת עצבית: ב־KNN ההצבעה המשוקללת מבוססת על מרחקים בזמן החיזוי, ולא על עדכון הדרגתי של וקטור משקולות באמצעות Gradient Descent.

14. קנה מידה ותקנון — בדיקת שגיאה מקצועית

מרחק אוקלידי רגיש ליחידות המדידה. אם מאפיין אחד נע בין 0 ל־10 והשני בין 0 ל־10,000, ההפרש המספרי במאפיין השני עלול לשלוט במרחק. לפיכך, ייתכן שנקודה תיראה רחוקה בעיקר בגלל סולם המדידה ולא בגלל חשיבות אמיתית של המאפיין. כבר למדנו תקנון בפרק 2.3; כאן ניישם אותו באופן מקצועי.



איור 8. הבדל גדול ביחידות של מאפיין יכול להשתלט על המרחק הכולל.

את ממוצע האימון וסטיית התקן של האימון יש לחשב רק על נתוני האימון. נתוני Test עוברים transform באמצעות אותם פרמטרים; אין להשתמש בהם כדי להתאים את הממיר. בוולידציה צולבת יש להתאים מחדש את הממיר בתוך כל קיפול אימון, ולכן Pipeline הוא כלי העבודה המועדף.

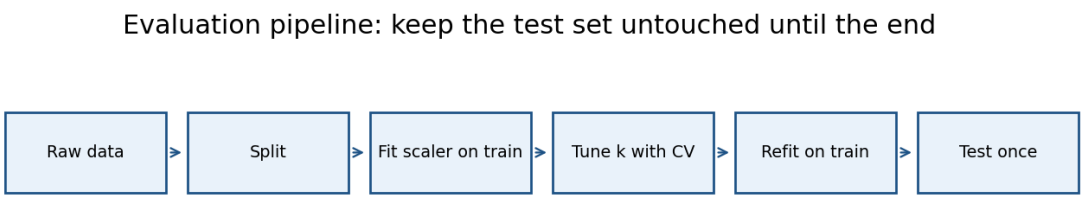
15. בניית ניסוי אמיתי על Iris בלי זליגת מידע

נעבור לדוגמה מקצועית מתוך Iris: קבוצת נתונים מוכרת בעלת ארבע תכונות מדידה ושלושה סוגי אירוסים. כאן נבחר לצורך גרפים שתי תכונות בלבד, ונשמור את יחס המחלקות בעת החלוקה. בנתוני פרויקט אמיתי חשוב לתאר גם מקור נתונים, איכותם, ערכים חסרים ומגבלות ייצוג האוכלוסייה.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=19, stratify=y
)
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
pipe.fit(X_train, y_train)
print("Test accuracy:", pipe.score(X_test, y_test))
```

למה לא להפעיל scaler.fit_transform על כל X לפני החלוקה? משום שאז ממוצעי נתוני הבדיקה ישפיעו על אופן הכנת נתוני האימון. זוהי זליגת מידע. שימו לב גם ש־pipe.fit אינו ״מלמד״ את KNN משקולות כמו ברשת עצבית; הוא מתאים את הממיר ושומר את דוגמאות האימון שהמסווג צריך בזמן החיזוי.

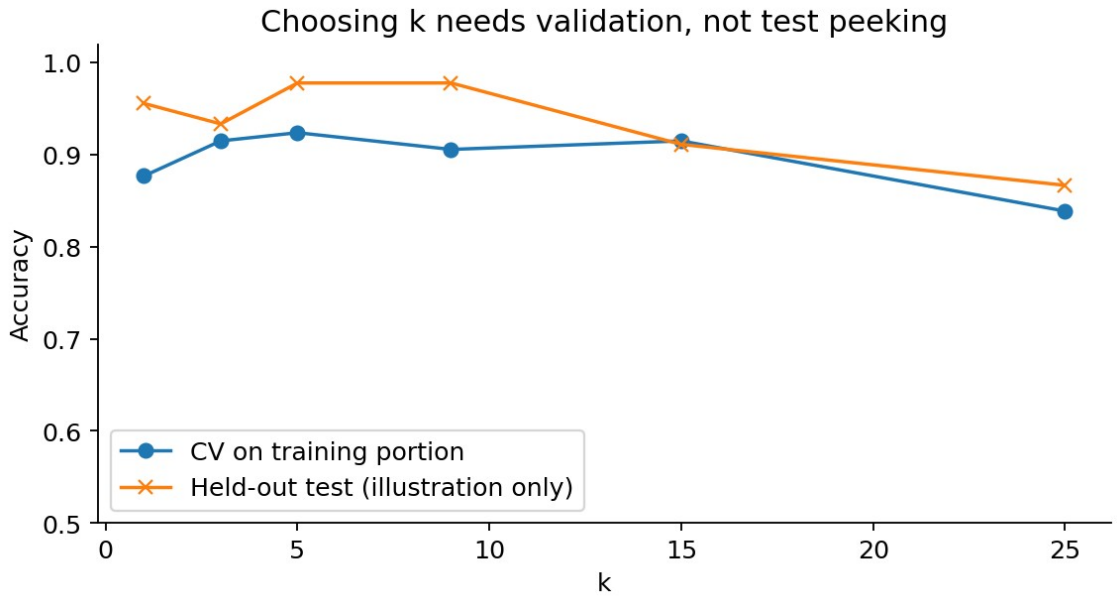


איור 9. סדר עבודה תקין: חלוקה, התאמת הכנה בתוך אימון, בחירת k בוולידציה ובדיקת Test בסוף.

16. בחירת k באמצעות Cross-Validation

היפר־פרמטר k משפיע על רגישות המודל. ערך קטן מאוד יכול לגרום להחלטה רגישה לרעש, וערך גדול מדי עשוי להחליק גבולות החלטה יתר על המידה. איננו בוחרים k לפי Test; בוחרים אותו באמצעות נתוני האימון, למשל באמצעות K-Fold או StratifiedKFold. קיפולים מספקים כמה אומדנים, אך אין לראות בממוצע שלהם הבטחה לתוצאה זהה בנתוני אמת.

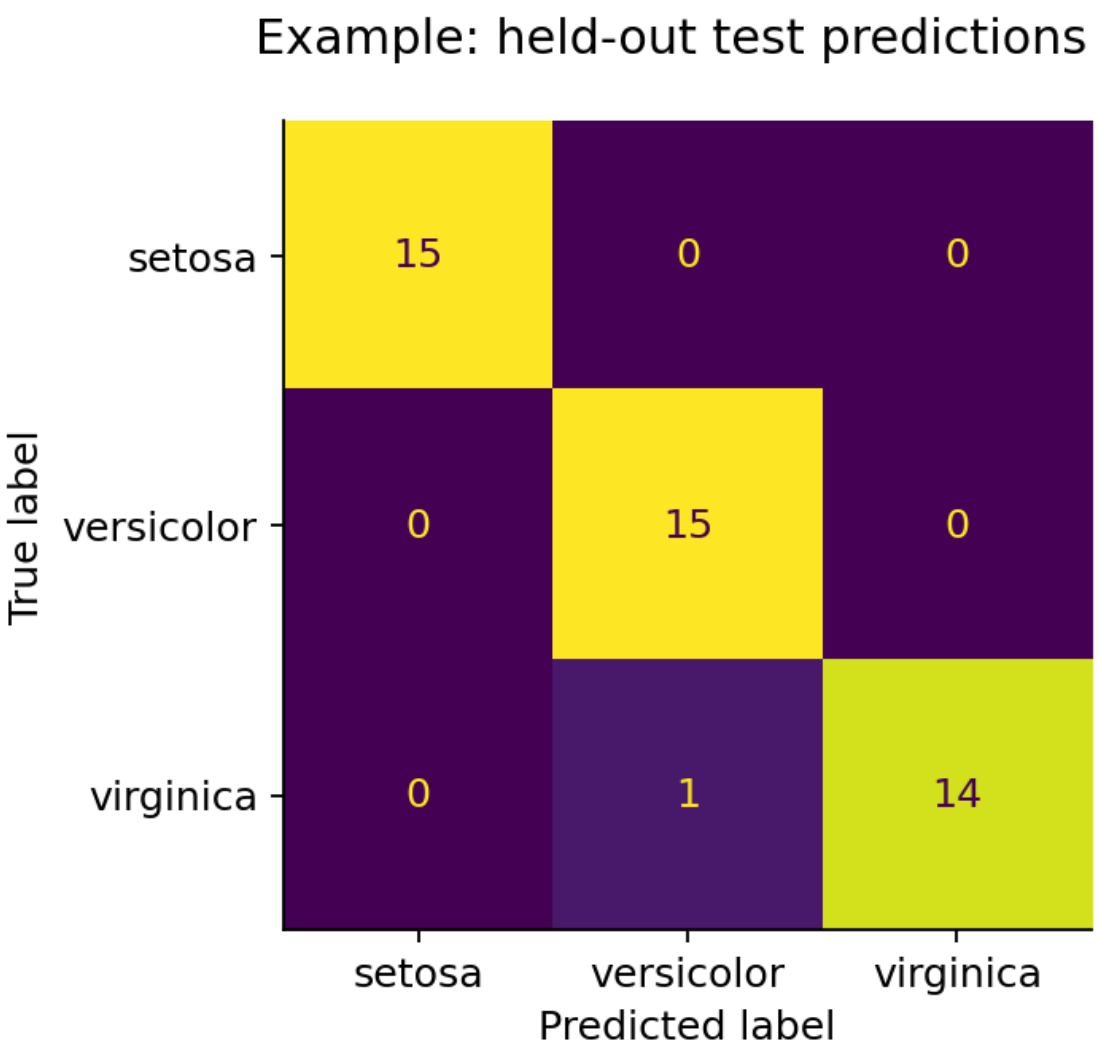
```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=19)
for k in [1, 3, 5, 9, 15]:
    candidate = Pipeline([
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    scores = cross_val_score(candidate, X_train, y_train,
                             cv=cv, scoring="accuracy")
    print(k, round(scores.mean(), 3), round(scores.std(), 3))
```



איור 10. דוגמה מדגמית על Iris: עקומות CV ו־Test עשויות להשתנות עם החלוקה. בוחרים k מתוך CV בלבד; Test מוצג לצורך דיון לאחר הבחירה.

17. מדדי ביצוע — מעבר למספר Accuracy בודד

לפעמים מודל מסווג היטב את המחלקה הגדולה אך מחמיץ מחלקה נדירה וחשובה. כדי להבין מה קרה נבחן מטריצת בלבול, Precision, Recall ו־F1 לפי הצורך. במקרה של כמה מחלקות נדווח גם כיצד אוחדו המדדים בין מחלקות — למשל macro או weighted — ולא רק מספר ממוצע חסר הסבר.



איור 11. דוגמה למטריצת בלבול מתוך קבוצת בדיקה מופרדת; האלכסון מציין תחזיות נכונות.

בחינת הבגרות דורשת יכולת להסביר תוצאה, ולא להסתפק בשורת פלט. אם מודל קיבל Accuracy גבוה, התלמיד צריך לדעת לשאול: בכמה דוגמאות נבדק? מה יחס המחלקות? האם התבצעה התאמת תקנון על נתוני הבדיקה? מהו מחיר טעות מסוג מסוים?

18. בדיקות יחידה למימוש KNN שלנו

לפני שמציגים פרויקט גמר בודקים גם מקרים קטנים שביכולתנו לחשב ביד. בדיקת יחידה טובה אינה דורשת מאגר גדול: לעיתים דווקא שלוש נקודות מאפשרות לזהות שגיאה. רצוי לתכנן מראש בדיקה למרחק אפס, לשתי נקודות שוות מרחק, לאינדקסי מיון, לממדים לא תואמים ולמקרה שבו k גדול ממספר דוגמאות האימון.

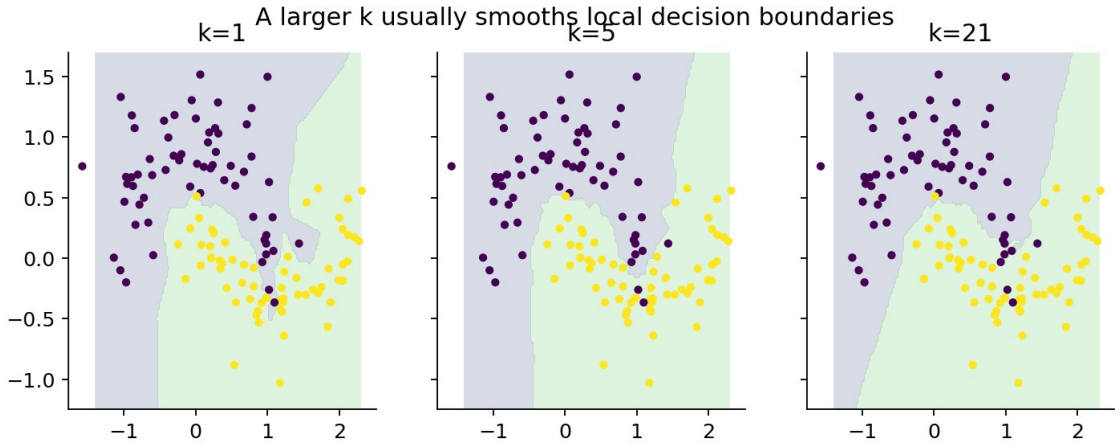
```
# Tests for our teaching implementation
assert predict_one(X=np.array([[0.,0.],[3.,0.]]),
                   y=np.array(["A", "B"]),
                   q=np.array([.2,0.]), k=1) == "A"

assert np.isclose(
    np.linalg.norm(np.array([3.,4.]) - np.array([0.,0.])), 5.0
)
print("Small deterministic tests passed")
```

חשוב להבדיל בין בדיקת נכונות לבין הערכת דיוק על מאגר: בדיקת יחידה אומרת שהמימוש פועל לפי הכללים שכתבנו במקרה ידוע; Accuracy על Test מודד כיצד המודל מכליל לדוגמאות חדשות. האחת אינה מחליפה את האחרת.

19. על אילו מקרים KNN עלול להיכשל?

KNN יכול להיכשל בשל בחירת מאפיינים לא מתאימים, קנה מידה, רעש, חוסר איזון במחלקות או מספר גדול של ממדים. במרחב עתיר ממדים, מרחקים בין נקודות עלולים להיות פחות מבחינים, תופעה המכונה לעיתים ״קללת הממדיות״. אין פירוש הדבר ש־KNN לעולם אינו עובד עם הרבה מאפיינים, אלא שיש לבדוק האם המרחק שנבחר משמעותי עבור המשימה והנתונים.



איור 12. שינוי k משפיע על צורת גבולות ההחלטה; הגבול החלק ביותר אינו בהכרח המתאים ביותר.

בנוסף, מאגר שיש בו שתי קבוצות של אותה מחלקה עשוי להיות מבלבל עבור Nearest Centroid, אך KNN יכול לעיתים לזהות את המבנה המקומי. מצד שני, כאשר יש מספר עצום של נקודות אימון ומערכת חייבת לענות במהירות רבה, העלויות של KNN יכולות להקשות על שימוש ישיר. בחירת המודל תמיד תלויה בבעיה.

20. כיצד מחברים KNN לעיבוד תמונה?

בפרק 6 למדנו שתמונה היא מערך מספרי. אפשר לשטח תמונת גווני אפור לווקטור ולהגדיר מרחק בין תמונות. אז לכאורה ניתן להפעיל KNN על פיקסלים: תמונה חדשה תסווג לפי התמונות הקרובות אליה. זהו קו בסיס לימודי שימושי, אך מרחק פיקסלים רגיש להזזה, תאורה וסיבוב, ולכן אינו מבטיח לזהות צורות סמנטיות דומות.

הקשר העמוק לפרקי CNN יהיה ברור בהמשך: במקום להשוות רק ערכי פיקסלים גולמיים, רשת עצבית יכולה ללמוד ייצוגים של מאפיינים חזותיים. לאחר לימוד רשתות נוכל לשאול אם KNN על ייצוגים נלמדים נותן תוצאות טובות יותר מ־KNN על פיקסלים. זו הרחבת גשר לפרויקט, ולא תחליף לפרק CNN.

21. יישום בפרויקט גמר — דף החלטות הנדסי

תלמיד שבוחר להשתמש ב־KNN בפרויקט צריך לתעד את שאלת המחקר, מקור הנתונים, שמות המאפיינים, יחידותיהם, דרך הטיפול בחסרים, אופן החלוקה, שיטת התקנון, רשימת ערכי k שנבדקו, מדדי ההערכה, מגבלות הביצועים והסיבה לבחירת המודל. יש להימנע מתיעוד טכני שנכתב לאחר מעשה ואינו משקף ניסוי שבוצע באמת.

מה צריך לנמק?	החלטה
האם הם זמינים בזמן חיזוי ובעלי משמעות?	מאפיינים
מדוע Euclidean או מדד אחר מתאים לנתונים?	מרחק
כיצד הותאם רק על Train?	תקנון
כיצד נבחר באמצעות Validation/CV?	k
מהו מחיר הטעות ומהי המחלקה החשובה?	מדד
מה עלול להשתבש בנתונים חדשים?	מגבלה

22. טעויות נפוצות שכדאי לעצור בזמן

- סידור distances במיון ושכחת לעדכן באותו סדר את התוויות המתאימות.
- שימוש ב־axis=0 במקום axis=1 בחישוב מרחקי שאילתה אחת, או במקום axis=2 בחישוב כמה שאילתות.
- הפעלת fit_transform על כל הדוגמאות לפני חלוקת Test או לפני CV.
- הנחה שהמימוש הידני ו־sklearn יכריעו כל שוויון באותה דרך.
- חשיבה ש־predict_proba בהכרח נותנת הסתברות מכוילת היטב לשימוש בהחלטה רגישה.
- שימוש ב־Accuracy אימון כהוכחה להכללה, או בחירת k לאחר הסתכלות ב־Test.
- בניית מטריצת Broadcasting ענקית בלי להבין את מחיר הזיכרון.
- תיאור k בתור פרמטר שנלמד מן הנתונים; k הוא היפר־פרמטר שהמפתח בוחר או מכוון.

23. סיכום מושגים, תפקידם והקשר ל־AI

מושג	מה צריך לדעת להסביר
shape	מה המשמעות של כל ציר במערך NumPy
Broadcasting	מתי ניתן לבצע פעולת מערך ללא לולאות Python מפורשות
argsort	כיצד לשמור קשר בין מרחק לאינדקס ולתווית
k	השפעת מספר השכנים על החלטת סיווג
Tie policy	מה קורה במקרה של שוויון מרחק או קולות

מה צריך לדעת להסביר	מושג
מדוע הכנת הנתונים חייבת להיכלל בתהליך האימון	Pipeline
כיצד בוחרים היפר־פרמטר בלי לגעת ב־Test	CV
מה מחיר מרחקים, מיון ואחסון בנתונים גדולים	Complexity

24. תרגול מדורג — ללא פתרונות

רמת יסוד: הבנת מבנה המימוש

- שאלה 1. נתונה מטריצה X בעלת 20 שורות ו־4 עמודות ושאילתה q בעלת 4 רכיבים. כתבו את shape של X, של q, של X-q ושל וקטור המרחקים.
- שאלה 2. לפניכם מרחקים [2.0,0.5,1.0] ותוויות [A,B,A]. קבעו אילו אינדקסים תחזיר argsort, ומה תהיה התחזית כאשר k=1 וכאשר k=3.
- שאלה 3. הסבירו במילים מדוע אי־אפשר פשוט למיין את מערך התוויות כאשר ממיינים את המרחקים.
- שאלה 4. תלמיד כתב np.sum((X-q)**2, axis=0). הסבירו מה הקוד מחשב ולמה זו אינה התוצאה הנדרשת.

רמת העמקה: מתמטיקה, ניסוי ותכנון

- שאלה 5. בנו מערכת של ארבע דוגמאות ושתי מחלקות שבה מספר הקולות ב־k=4 שווה. הגדירו כלל הכרעה ודונו בהשפעתו.
- שאלה 6. נתון מאגר עם 2,000 דוגמאות ו־30 מאפיינים. העריכו כמה פעולות הבדל־ריבוע נדרשות לשאילתה אחת במימוש brute-force.
- שאלה 7. הסבירו מדוע שימוש ב־StandardScaler לפני cross_val_score יכול לגרום לזליגת מידע, והציעו תיקון באמצעות Pipeline.
- שאלה 8. תכננו השוואה בין k=1, k=5 ו־k=15 על Iris שתשתמש ב־CV לבחירה וב־Test להערכה סופית בלבד.

רמת בגרות ופרויקט: נימוק מקצועי

- שאלה 9. תלמיד טוען כי ״כל מימוש NumPy מהיר יותר כי אין בו לולאות״. הסבירו מדוע הטענה גורפת מדי תוך התייחסות לזיכרון.
- שאלה 10. מערכת לזיהוי חריגות בנתוני תלמידים מקבלת Accuracy של 97%, אך כמעט אינה מזהה את המחלקה הנדירה. הציעו לפחות שני מדדים או ניסויים נוספים.
- שאלה 11. שני מימושים של KNN מחזירים תוצאה שונה רק כאשר השכן ה־k והשכן ה־k+1 נמצאים באותו מרחק. הציעו בדיקת יחידה והסבירו מקור אפשרי להבדל.
- שאלה 12. כתבו פסקת הגנה בעל־פה: מדוע בחרתם ב־KNN, מה יתרונו, מה חולשתו ומה תרצו לשנות אם המאגר יגדל פי מאה.

25. משימת סיכום — ממימוש למערכת קטנה

בנו מחברת Colab המשווה בין המימוש העצמאי שלכם לבין KNeighborsClassifier על אותה חלוקת נתונים. צרו טבלת השוואה של חיזויים, נתחו מקרים שבהם התחזיות שונות, בדקו תחילה את מדיניות השוויון, והשוו מדדי בדיקה. לאחר מכן כתבו דוח קצר שבו מוסברים מקורות הנתונים, השיטה, החלטת k, שיטת התקנון, מגבלה אחת והמלצה להמשך. הפעילו את כל התאים מהתחלה לפי סדרם כדי לוודא שחזוריות.

26. מפת הקשרים בין שלוש תכניות הלימודים

מעמד	מה נלמד או נדרש	מסלול
ידע קודם	הפעלה והבנה ראשונית של KNN ומדדי ביצוע	יחידה 3 — בסיס
עוגן בליבה	מרחקים, גבולות החלטה, k, ניתוח וסיווג	יחידה 5 — נושאי ליבה
עוגן התמחות	מימוש עצמי, NumPy, Broadcasting וניסויים על Iris	התמחות DL — פרק 7
העמקה דידקטית	בדיקות יחידה, זיכרון, מגבלות ומהירות	הרחבת פרויקט

27. קריאה משלימה ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN, 30 שעות (לפי מפת המאגר). מצגות ההשתלמות KNN 5.1, KNN 5.2 ו־KNN with sklearn 5.3 KNN with code implementation; מחברות Iris ותרגולי המימוש העצמאי.

ספרי יסוד ששיטות ההסבר שלהם משמשות השראה לכתיבה מקורית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; Deisenroth, Faisal & Ong, Mathematics for Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Zhang et al., Dive into Deep Learning.

תיעוד טכני: https://numpy.org/doc/stable/user/basics.broadcasting ; https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html ; https://scikit-learn.org/stable/modules/cross_validation.html

הערה: ההסברים, הדוגמאות, הטבלאות והאיורים נכתבו ויוצרו לפרק זה באופן מקורי. התאמה לנוסח המחייב של כל סעיף בבגרות טעונה בדיקה מול מסמכי משרד החינוך התקפים לשנת הלימודים.